



The author: Georgi 'Sanmayce' Marinov, rendered by PixAI.art from an original photo



FASTEST LOOKUP HASH

```
// https://godbolt.org/z/o3ooEnToM
// x86-64 clang 22.1.0
// -O3 -maes
/*
.LCPI0_0:
.quad -7046029254386353131
.quad 7809847782465536322
.LCPI0_1:
.quad -4942790177534073029
.quad 7640891576956012808
.LCPI0_2:
.quad 4794697086780616226
.quad 4354685564936845355
.LCPI0_3:
.quad 7809847782465536322
.quad -7046029254386353131
.LCPI0_4:
.quad 7640891576956012808
.quad -4942790177534073029
.LCPI0_5:
.quad 4354685564936845355
.quad 4794697086780616226
FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla(char
const*, unsigned long, unsigned int, void*):
mov r8, rcx
mov eax, esi
xor eax, edx
movd xmm0, eax
pshufd xmm0, xmm0, 0
cmp rsi, 9
jb .LBB0_9
cmp rsi, 17
jb .LBB0_8
lea rcx, [rsi - 1]
mov rdx, rcx
shr rdx, 5
lea rax, [rdx + 1]
test cl, 32
jne .LBB0_5
shl rax, 4
mov rcx, rsi
sub rcx, rax
add rcx, rdi
shl rdx, 4
add rdx, 16
movdqa xmm1, xmmword ptr [rip + .LCPI0_0]
movdqa xmm2, xmmword ptr [rip + .LCPI0_1]
movdqa xmm3, xmmword ptr [rip + .LCPI0_2]
xor eax, eax
.LBB0_4:
movdqu xmm4, xmmword ptr [rdi + rax]
movdqu xmm5, xmmword ptr [rcx + rax]
aesenc xmm1, xmm4
```

```
aesenc xmm2, xmm5
aesenc xmm3, xmm4
aesenc xmm3, xmm5
add rax, 16
cmp rdx, rax
jne .LBB0_4
jmp .LBB0_10
.LBB0_9:
mov rax, qword ptr [rdi]
lea ecx, [8*rsi]
neg cl
shl rax, cl
movq xmm1, rax
shr rax, cl
movq xmm4, rax
movdqa xmm5, xmm1
pumpckldq xmm5, xmm4
pumpckldq xmm4, xmm1
movdqa xmm1, xmmword ptr [rip + .LCPI0_0]
aesenc xmm1, xmm5
movdqa xmm2, xmmword ptr [rip + .LCPI0_1]
aesenc xmm2, xmm4
movdqa xmm3, xmmword ptr [rip + .LCPI0_2]
aesenc xmm3, xmm5
aesenc xmm3, xmm4
jmp .LBB0_10
.LBB0_8:
movq xmm1, qword ptr [rdi]
pshufd xmm2, xmm1, 68
movq xmm1, qword ptr [rdi + rsi - 8]
pshufd xmm3, xmm1, 68
pxor xmm2, xmm0
pxor xmm3, xmm0
movdqa xmm1, xmmword ptr [rip + .LCPI0_0]
aesenc xmm1, xmm2
movdqa xmm2, xmmword ptr [rip + .LCPI0_1]
aesenc xmm2, xmm3
movdqa xmm3, xmmword ptr [rip + .LCPI0_2]
jmp .LBB0_10
.LBB0_5:
shr rax
shl rdx, 4
mov rcx, rsi
sub rcx, rdx
movdqa xmm1, xmmword ptr [rip + .LCPI0_0]
movdqa xmm2, xmmword ptr [rip + .LCPI0_1]
movdqa xmm3, xmmword ptr [rip + .LCPI0_2]
movdqa xmm6, xmmword ptr [rip + .LCPI0_3]
movdqa xmm5, xmmword ptr [rip + .LCPI0_4]
movdqa xmm4, xmmword ptr [rip + .LCPI0_5]
.LBB0_6:
movdqu xmm7, xmmword ptr [rdi]
movdqu xmm8, xmmword ptr [rdi + 16]
```

```
movdqu xmm9, xmmword ptr [rdi + rcx - 16]
movdqu xmm10, xmmword ptr [rdi + rcx]
add rdi, 32
aesenc xmm1, xmm7
aesenc xmm6, xmm8
aesenc xmm2, xmm9
aesenc xmm5, xmm10
aesenc xmm3, xmm7
aesenc xmm4, xmm8
aesenc xmm3, xmm9
aesenc xmm4, xmm10
dec rax
jne .LBB0_6
aesenc xmm0, xmm6
aesenc xmm0, xmm5
aesenc xmm0, xmm4
.LBB0_10:
aesenc xmm0, xmm1
aesenc xmm0, xmm2
aesenc xmm0, xmm3
movq rcx, xmm0
pshufd xmm0, xmm0, 238
movq rax, xmm0
mul rcx
xor esi, edx
xor esi, eax
mov dword ptr [r8], esi
ret
```


Das Tanzlied des Totenschiffes

Was gehn euch meine Lumpen an?
Da hängen Freud' und Tränen dran.
Was kümmert euch denn mein Gesicht?
Ich brauche euer Mitleid nicht.

Was kümmert euch, was mir gefällt?
Ich lebe mich, nicht euch, in dieser Welt.
In euren Himmel will ich gar nicht 'rein,
viel lieber dann schon in der Hölle sein.

Ich brauch' gewiß nicht eure Gnaden,
und selbst wenn Tote ich geladen,
wenn Schimpf und Schand' sind an mir dran,
euch geht das einen Sch...dreck an.

Ich pfeife auf das Weltgericht.
An Auferstehung glaub' ich nicht,
ob's Götter gibt, das weiß ich nicht,
und Höllenstrafen fürcht' ich nicht!

Hoppla he, auf weiter See,
hoppla, hoppla, he!



P
I
P
P
I
P

Das Tanzlied des Totenschiffes

Was gehn euch meine Lumpen an?
Da hängen Freud' und Tränen dran.
Was kümmert euch denn mein Gesicht?
Ich brauche euer Mitleid nicht.

Was kümmert euch, was mir gefällt?
Ich lebe mich, nicht euch, in dieser Welt.
In euren Himmel will ich gar nicht 'rein,
viel lieber dann schon in der Hölle sein.

Ich brauch' gewiß nicht eure Gnaden,
und selbst wenn Tote ich geladen,
wenn Schimpf und Schand' sind an mir dran,
euch geht das einen Sch...dreck an.

Ich pfeife auf das Weltgericht.
An Auferstehung glaub' ich nicht,
ob's Götter gibt, das weiß ich nicht,
und Höllenstrafen fürcht' ich nicht.

Hoppla he, auf weiter See,
hoppla, hoppla, he!



P
I
P
P
I
P

Das Tanzlied des Totenschiffes

Was gehn euch meine Lumpen an?
Da hängen Freud' und Tränen dran.
Was kümmert euch denn mein Gesicht?
Ich brauche euer Mitleid nicht.

Was kümmert euch, was mir gefällt?
Ich lebe mich, nicht euch, in dieser Welt.
In euren Himmel will ich gar nicht 'rein,
viel lieber dann schon in der Hölle sein.

Ich brauch' gewiß nicht eure Gnaden,
und selbst wenn Tote ich geladen,
wenn Schimpf und Schand' sind an mir dran,
euch geht das einen Sch...dreck an.

Ich pfeife auf das Weltgericht.
An Auferstehung glaub' ich nicht,
ob's Götter gibt, das weiß ich nicht,
und Höllenstrafen fürcht' ich nicht.

Hoppla he, auf weiter See,
hoppla, hoppla, he!

Ɔ
I
Ɔ
Ɔ
I
Ɔ



Das Tanzlied des Totenschiffes

Was gehn euch meine Lumpen an?
Da hängen Freud' und Tränen dran.
Was kümmert euch denn mein Gesicht?
Ich brauche euer Mitleid nicht.

Was kümmert euch, was mir gefällt?
Ich lebe mich, nicht euch, in dieser Welt.
In euren Himmel will ich gar nicht 'rein,
viel lieber dann schon in der Hölle sein.

Ich brauch' gewiß nicht eure Gnaden,
und selbst wenn Tote ich geladen,
wenn Schimpf und Schand' sind an mir dran,
euch geht das einen Sch...dreck an.

Ich pfeife auf das Weltgericht.
An Auferstehung glaub' ich nicht,
ob's Götter gibt, das weiß ich nicht,
und Höllenstrafen fürcht' ich nicht.

Hoppla he, auf weiter See,
hoppla, hoppla, he!

パイパイ



Das Tanzlied des Totenschiffes

Was gehn euch meine Lumpen an?
Da hängen Freud' und Tränen dran.
Was kümmert euch denn mein Gesicht?
Ich brauche euer Mitleid nicht.

Was kümmert euch, was mir gefällt?
Ich lebe mich, nicht euch, in dieser Welt.
In euren Himmel will ich gar nicht 'rein,
viel lieber dann schon in der Hölle sein.

Ich brauch' gewiß nicht eure Gnaden,
und selbst wenn Tote ich geladen,
wenn Schimpf und Schand' sind an mir dran,
euch geht das einen Sch...dreck an.

Ich pfeife auf das Weltgericht.
An Auferstehung glaub' ich nicht,
ob's Götter gibt, das weiß ich nicht,
und Höllenstrafen fürcht' ich nicht.

Hoppla he, auf weiter See,
hoppla, hoppla, he!

パイパイ





The author: Georgi 'Sanmayce' Marinov, rendered by PixAI.art from an original photo



// FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla: the 100% FREE lookuper, last update: 2026-Aug-09, Kaze (sanmayce@sanmayce.com). [
// This latest revision was written when Mikayla "saveafox" left this world.

/*

FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla (FOX Variant)

The Fastest Lookuper. Engineered for maximum sequential throughput.

DESCRIPTION:

A brutally fast, non-incremental, 128-bit SIMD vector hash built for modern x86-64 architectures. Designed to saturate out-of-order (OOO) execution pipelines using AES-NI hardware acceleration.

VALIDATION:

SMHasher3 Verified (Passed).

WARNING:

Built for speed, not standard compliance. Short-key processing (0-8 bytes) intentionally disregards out-of-bounds memory read safety to prioritize absolute minimum cycle-counts.

*/

```
// "There it now stands for ever. Black on white.
// I can't get away from it. Ahoy, Yorikke, ahoy, hoy, ho!
// Go to hell now if you wish. What do I care? It's all the same now to me.
// I am part of you now. Where you go I go, where you leave I leave, when you go to the devil I go. Married.
// Vanished from the living. Damned and doomed. Of me there is not left a breath in all the vast world.
// Ahoy, Yorikke! Ahoy, hoy, ho!
// I am not buried in the sea,
// The death ship is now part of me
// So far from sunny New Orleans
// So far from lovely Louisiana."
// /An excerpt from 'THE DEATH SHIP - THE STORY OF AN AMERICAN SAILOR' by B.TRAVEN/
//
// "Walking home to our good old Yorikke, I could not help thinking of this beautiful ship, with a crew on board that had faces as if they
were seeing ghosts by day and by night.
// Compared to that gilded Empress, the Yorikke was an honorable old lady with lavender sachets in her drawers.
// Yorikke did not pretend to anything she was not. She lived up to her looks. Honest to her lowest ribs and to the leaks in her bilge.
// Now, what is this? I find myself falling in love with that old jane.
// All right, I cannot pass by you, Yorikke; I have to tell you I love you. Honest, baby, I love you.
// I have six black finger-nails, and four black and green-blue nails on my toes, which you, honey, gave me when necking you.
// Grate-bars have crushed some of my toes. And each finger-nail has its own painful story to tell.
// My chest, my back, my arms, my legs are covered with scars of burns and scorchings.
// Each scar, when it was being created, caused me pains which I shall surely never forget.
// But every outcry of pain was a love-cry for you, honey.
// You are no hypocrite. Your heart does not bleed tears when you do not feel heart-aches deeply and truly.
// You do not dance on the water if you do not feel like being jolly and kicking chasers in the pants.
// Your heart never lies. It is fine and clean like polished gold. Never mind the rags, honey dear.
// When you laugh, your whole soul and all your body is laughing.
// And when you weep, sweetly, then you weep so that even the reefs you pass feel like weeping with you.
// I never want to leave you again, honey. I mean it. Not for all the rich and elegant buckets in the world.
// I love you, my gypsy of the sea!"
// /An excerpt from 'THE DEATH SHIP - THE STORY OF AN AMERICAN SAILOR' by B.TRAVEN/
//
// Dedicated to Pippip, the main character in the 'Das Totenschiff' roman, actually the B.Traven himself, his real name was Hermann Albert
Otto Maksymilian Feige.
// CAUTION: Add 8 more bytes to the buffer being hashed, usually malloc(...+8) - to prevent out of boundary reads!
// Many thanks go to Yurii 'Hordi' Hordiienko, he lessened with 3 instructions the original 'Pippip', thus:
//
```

#include <stdlib.h>

Listing: FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla_FOX; Update: 2026-Aug-10, Kaze(sanmayce@sanmayce.com);


```

#include <stdint.h>
#include <immintrin.h>
#define HashInBITS 32 // default 26 i.e. 2^26 i.e. 64MS(Mega Slots); slots contain 8bytes pointers or 512MB, because many netbooks have 512MB
free (1GB in total)!
#define MASK ((1ULL << HashInBITS) - 1)
#define _PADr_KAZE(x, n) ( ((x) << (n)) >> (n) )
#define _PAD_KAZE(x, n) ( ((x) << (n)) )

typedef struct {
    uint64_t low;
    uint64_t high;
} hashPippip128_t;

static inline uint64_t fold64(uint64_t A, uint64_t B) {
    #if defined(_GNUCC) || defined(_clang_)
        uint128_t r = (_uint128_t)A * B;
        return (uint64_t)r ^ (uint64_t)(r >> 64);
    #else
        uint64_t hash64 = A ^ B;
        hash64 *= 1099511628211; //591798841;
        return hash64;
    #endif
}

// Actually 'FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla_FOX'
void FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla (const char *str, size_t wrdlen, uint32_t seed, void *output) {
    __m128i chunkA;
    __m128i chunkB;
    __m128i stateMIX;
    uint64_t hashLH;
    uint64_t hashRH;

    // ### 1. '0x6C62272E07BB0142' (The FNV-128 Offset Basis)
    //
    // * * *Origin: * * * This is the upper 64 bits of the * * *128-bit FNV offset basis* * *.
    // * * *Derivation: * * * The full 128-bit FNV offset basis is a massive, mathematically derived integer
    // ('144066263297769815596495629667062367629'), which in hexadecimal is '0x6C62272E07BB014262B821756295C58D'. The first half is
    // '0x6C62272E07BB0142'.
    // * * *Use Case: * * * It acts as the seed/starting state for Fowler-Noll-Vo (FNV) 128-bit non-cryptographic hashes.
    // ### 2. '0x9E3779B97F4A7C15' (The Golden Ratio Constant)
    // * * *Origin: * * * This is derived from the * * *Golden Ratio* * * ($\phi$).
    // * * *Derivation: * * * It represents the fractional part of $\phi \times 2^{64}$. Specifically, $2^{64} / \phi \approx 11400714819323198485$,
    // which translates directly to '0x9E3779B97F4A7C15' in 64-bit hexadecimal.
    // * * *Name/Use Case: * * * Widely known as the * * *RC5 Magic Constant* * * ($Q_{64}$), the * * *Golden Gamma* * *, or 'pow64_over_phi'. It is incredibly
    // popular for scattering bits cleanly, featuring heavily in SplitMix64 pseudo-random number generators, the Tiny Encryption Algorithm (TEA),
    // and SpookyHash.

    // * * '0x6A09E667F3BCC908' - Fractional part of $\sqrt{2}$. (SHA-512 IV 0)
    // * * '0xBB67AE8584CAA73B' - Fractional part of $\sqrt{3}$. (SHA-512 IV 1)
    // * * '0x3C6EF372FE94F82B' - Fractional part of $\sqrt{5}$. (SHA-512 IV 2)
    // * * '0x428A2F98D728AE22' - Fractional part of $\sqrt{3}[2]$. (SHA-512 Round Constant 0)
    //
    // * * *2. Pi and Euler's Number (The Blowfish / RC5 Companions) * * *
    //
    // * * '0x243F6A8885A308D3' - Derived from the fractional digits of * * $\pi$ * * (Pi). Famous for being the initialization vector for the Blowfish
    // cipher.
    // * * '0xB7E151628AED2A6B' - Derived from the fractional digits of * * $e$ * * (Euler's number). This is the companion constant ($P_{64}$) used
    // right alongside the Golden Ratio constant in the RC5 cipher initialization.

    stateMIX = _mm_set1_epi32( (uint32_t)wrdlen ^ seed );
    __m128i stateA = _mm_set_epi64x(0x6c62272e07bb0142, 0x9e3779b97f4a7c15);
    __m128i stateB = _mm_set_epi64x(0x6a09e667f3bcc908, 0xbb67ae8584caa73b);
    __m128i stateC = _mm_set_epi64x(0x3c6ef372fe94f82b, 0x428a2f98d728ae22);

    if (wrdlen > 8) {
        size_t Cycles, NDhead;
        if (wrdlen > 16) {
            Cycles = ((wrdlen - 1) >> 5) + 1;
            NDhead = wrdlen - (Cycles << 4);
            if (Cycles & 1) {
                #pragma nounroll
                for(; Cycles--; str += 16) {
                    chunkA = _mm_loadu_si128((__m128i *) (str));
                    chunkB = _mm_loadu_si128((__m128i *) (str+NDhead));
                    stateA = _mm_aesenc_si128(stateA, chunkA);
                    stateB = _mm_aesenc_si128(stateB, chunkB);
                    stateC = _mm_aesenc_si128(stateC, chunkA);
                    stateC = _mm_aesenc_si128(stateC, chunkB);
                }
            } else {
                Cycles = Cycles >> 1;
                __m128i stateA2 = _mm_set_epi64x(0x9e3779b97f4a7c15, 0x6c62272e07bb0142);
                __m128i stateB2 = _mm_set_epi64x(0xbb67ae8584caa73b, 0x6a09e667f3bcc908);
                __m128i stateC2 = _mm_set_epi64x(0x428a2f98d728ae22, 0x3c6ef372fe94f82b);
                #pragma nounroll
                for(; Cycles--; str += 32) {
                    _mm_prefetch(str, _MM_HINT_T0);
                    _mm_prefetch(str+NDhead, _MM_HINT_T0);

                    chunkA = _mm_loadu_si128((__m128i *) (str));
                    __m128i chunkA2 = _mm_loadu_si128((__m128i *) (str+16));
                    stateA = _mm_aesenc_si128(stateA, chunkA);
                    stateA2 = _mm_aesenc_si128(stateA2, chunkA2);
                    chunkB = _mm_loadu_si128((__m128i *) (str+NDhead));
                    __m128i chunkB2 = _mm_loadu_si128((__m128i *) (str+NDhead+16));
                    stateB = _mm_aesenc_si128(stateB, chunkB);
                    stateB2 = _mm_aesenc_si128(stateB2, chunkB2);

                    stateC = _mm_aesenc_si128(stateC, chunkA);
                }
            }
        }
    }
}

```



```

        stateC2 = _mm_aesenc_si128(stateC2, chunkA2);

        stateC = _mm_aesenc_si128(stateC, chunkB);
        stateC2 = _mm_aesenc_si128(stateC2, chunkB2);
    }
    stateMIX = _mm_aesenc_si128(stateMIX, stateA2);
    stateMIX = _mm_aesenc_si128(stateMIX, stateB2);
    stateMIX = _mm_aesenc_si128(stateMIX, stateC2);
}
} else { // 9..16
    NDhead = wrdlen - (1<<3);
    hashLH = (*(uint64_t *) (str));
    hashRH = (*(uint64_t *) (str+NDhead));

    chunkA = _mm_set_epi64x(hashLH, hashLH);
    chunkB = _mm_set_epi64x(hashRH, hashRH);

    chunkA = _mm_xor_si128(chunkA, stateMIX);
    chunkB = _mm_xor_si128(chunkB, stateMIX);

    stateA = _mm_aesenc_si128(stateA, chunkA);
    stateB = _mm_aesenc_si128(stateB, chunkB);
}
} else {
    hashLH = _PADr_KAZE(*(uint64_t *) (str+0), (8-wrdlen)<<3); // wrdlen == 0 is not handled since <<64 is undefined
    hashRH = _PADr_KAZE(*(uint64_t *) (str+0), (8-wrdlen)<<3);
    chunkA = _mm_set_epi64x(hashLH, hashRH);
    chunkB = _mm_set_epi64x(hashRH, hashLH);
    stateA = _mm_aesenc_si128(stateA, chunkA);
    stateB = _mm_aesenc_si128(stateB, chunkB);
    stateC = _mm_aesenc_si128(stateC, chunkA);
    stateC = _mm_aesenc_si128(stateC, chunkB);
}

    stateMIX = _mm_aesenc_si128(stateMIX, stateA);
    stateMIX = _mm_aesenc_si128(stateMIX, stateB);
    stateMIX = _mm_aesenc_si128(stateMIX, stateC);

#ifdef eXdupe
    _mm_storeu_si128((__m128i *) output, stateMIX); // For eXdupe
#else
    uint64_t result[2];
    _mm_storeu_si128((__m128i *) result, stateMIX);
    uint64_t hash64 = fold64(result[0], result[1]);
    *(uint32_t *) output = (uint32_t) hash64 ^ wrdlen;
#endif
}

//      Cycles = ((wrdlen - 1)>>5) + 1;
//      NDhead = wrdlen - (Cycles<<4);
// And some visualization for XMM-WORD:
/**
kl= 33..64 Cycles= (kl-1)/32+1=2; MARGINAL CASES:
    2nd head starts at 33-2*16=1 or:
        0123456789012345 0123456789012345 0
    Head1: [XMM-WORD      ] [XMM-WORD      ]
    Head2: [XMM-WORD      ] [XMM-WORD      ]

    2nd head starts at 64-2*16=32 or:
        0123456789012345 0123456789012345 0123456789012345 0123456789012345
    Head1: [XMM-WORD      ] [XMM-WORD      ]
    Head2:                                     [XMM-WORD      ] [XMM-WORD      ]

kl=65..96 Cycles= (kl-1)/32+1=3; MARGINAL CASES:
    2nd head starts at 65-3*16=17 or:
        0123456789012345 0123456789012345 0123456789012345 0123456789012345 0
    Head1: [XMM-WORD      ] [XMM-WORD      ] [XMM-WORD      ]
    Head2:                                     [XMM-WORD      ] [XMM-WORD      ] [XMM-WORD      ]

    2nd head starts at 96-3*16=48 or:
        0123456789012345 0123456789012345 0123456789012345 0123456789012345 0123456789012345 0123456789012345
    Head1: [XMM-WORD      ] [XMM-WORD      ] [XMM-WORD      ]
    Head2:                                     [XMM-WORD      ] [XMM-WORD      ] [XMM-WORD      ]

*/

// And some visualization for Q-WORD:
/**
kl= 9..16 Cycles= (kl-1)/16+1=1; MARGINAL CASES:
    2nd head starts at 9-1*8=1 or:
        012345678
    Head1: [Q-WORD]
    Head2: [Q-WORD]

    2nd head starts at 16-1*8=8 or:
        0123456789012345
    Head1: [Q-WORD]
    Head2: [Q-WORD]

kl=17..24 Cycles= (kl-1)/16+1=2; MARGINAL CASES:
    2nd head starts at 17-2*8=1 or:
        01234567890123456
    Head1: [Q-WORD][Q-WORD]
    Head2: [Q-WORD][Q-WORD]

    2nd head starts at 24-2*8=8 or:
        012345678901234567890123
    Head1: [Q-WORD][Q-WORD]

```



```

Head2: [Q-WORD][Q-WORD]

k1=25..32 Cycles= (k1-1)/16+1=2; MARGINAL CASES:
2nd head starts at 25-2*8=9 or:
0123456789012345678901234
Head1: [Q-WORD][Q-WORD]
Head2: [Q-WORD][Q-WORD]

2nd head starts at 32-2*8=16 or:
01234567890123456789012345678901
Head1: [Q-WORD][Q-WORD]
Head2: [Q-WORD][Q-WORD]

k1=33..40 Cycles= (k1-1)/16+1=3; MARGINAL CASES:
2nd head starts at 33-3*8=9 or:
012345678901234567890123456789012
Head1: [Q-WORD][Q-WORD][Q-WORD]
Head2: [Q-WORD][Q-WORD][Q-WORD]

2nd head starts at 40-3*8=16 or:
0123456789012345678901234567890123456789
Head1: [Q-WORD][Q-WORD][Q-WORD]
Head2: [Q-WORD][Q-WORD][Q-WORD]

k1=41..48 Cycles= (k1-1)/16+1=3; MARGINAL CASES:
2nd head starts at 41-3*8=17 or:
01234567890123456789012345678901234567890
Head1: [Q-WORD][Q-WORD][Q-WORD]
Head2: [Q-WORD][Q-WORD][Q-WORD]

2nd head starts at 48-3*8=24 or:
01234567890123456789012345678901234567
Head1: [Q-WORD][Q-WORD][Q-WORD]
Head2: [Q-WORD][Q-WORD][Q-WORD]

*/

// The more the merrier, therefore I added the 10,000 GitHub stars performer xxhash also:
// https://github.com/Cyan4973/xxHash/issues/1029
//
// Pippip is not an extremely fast hash, it is the spirit of the author materialized disregarding anything outside the "staying true to
// oneself", or as one bona fide man Otto/Pippip once said:
//
// Translate as verbatim as possible:
// In 1926, Traven wrote that the only biography of a writer should be his
// works: «Die Biographie eines schöpferischen Menschen ist ganz und gar unwichtig.
// Wenn der Mensch in seinen Werken nicht zu erkennen ist, dann ist entweder der
// Mensch nichts wert oder seine Werke sind nichts wert. Darum sollte der schöpferische
// Mensch keine andere Biographie haben als seine Werke» (Hauschild, B. Traven: Die
// unbekannten Jahre, op. cit., p. 31.)
//
// In 1926, Traven wrote that the only biography of a writer should be his works:
// "The biography of a creative person is completely and utterly unimportant.
// If the person is not recognizable in his works, then either the person is worthless or his works are worthless.
// Therefore, the creative person should have no other biography than his works" (Hauschild, B. Traven: Die unbekannten Jahre, op. cit., p.
// 31.)

// https://godbolt.org/z/o3ooEnToM
// x86-64 clang 22.1.0
// -O3 -maes
//
.LCPI0_0:
.quad -7046029254386353131
.quad 7809847782465536322
.LCPI0_1:
.quad -4942790177534073029
.quad 7640891576956012808
.LCPI0_2:
.quad 4794697086780616226
.quad 4354685564936845355
.LCPI0_3:
.quad 7809847782465536322
.quad -7046029254386353131
.LCPI0_4:
.quad 7640891576956012808
.quad -4942790177534073029
.LCPI0_5:
.quad 4354685564936845355
.quad 4794697086780616226
FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla(char const*, unsigned long, unsigned int, void*):
mov r8, rcx
mov eax, esi
xor eax, edx
movd xmm0, eax
pshufd xmm0, xmm0, 0
cmp rsi, 9
jb .LBB0_9
cmp rsi, 17
jb .LBB0_8
lea rcx, [rsi - 1]
mov rdx, rcx
shr rdx, 5
lea rax, [rdx + 1]
test cl, 32
jne .LBB0_5
shl rax, 4
mov rcx, rsi
sub rcx, rax
add rcx, rdi

```

```

    shl     rdx, 4
    add     rdx, 16
    movdqa  xmm1, xmmword ptr [rip + .LCPI0_0]
    movdqa  xmm2, xmmword ptr [rip + .LCPI0_1]
    movdqa  xmm3, xmmword ptr [rip + .LCPI0_2]
    xor     eax, eax
.LBB0_4:
    movdqu  xmm4, xmmword ptr [rdi + rax]
    movdqu  xmm5, xmmword ptr [rcx + rax]
    aesenc  xmm1, xmm4
    aesenc  xmm2, xmm5
    aesenc  xmm3, xmm4
    aesenc  xmm3, xmm5
    add     rax, 16
    cmp     rdx, rax
    jne     .LBB0_4
    jmp     .LBB0_10
.LBB0_9:
    mov     rax, qword ptr [rdi]
    lea     ecx, [8*rsi]
    neg     cl
    shl     rax, cl
    movq    xmm1, rax
    shr     rax, cl
    movq    xmm4, rax
    movdqa  xmm5, xmm1
    punpckldq  xmm5, xmm4
    punpckldq  xmm4, xmm1
    movdqa  xmm1, xmmword ptr [rip + .LCPI0_0]
    aesenc  xmm1, xmm5
    movdqa  xmm2, xmmword ptr [rip + .LCPI0_1]
    aesenc  xmm2, xmm4
    movdqa  xmm3, xmmword ptr [rip + .LCPI0_2]
    aesenc  xmm3, xmm5
    aesenc  xmm3, xmm4
    jmp     .LBB0_10
.LBB0_8:
    movq    xmm1, qword ptr [rdi]
    pshufd  xmm2, xmm1, 68
    movq    xmm1, qword ptr [rdi + rsi - 8]
    pshufd  xmm3, xmm1, 68
    pxor    xmm2, xmm0
    pxor    xmm3, xmm0
    movdqa  xmm1, xmmword ptr [rip + .LCPI0_0]
    aesenc  xmm1, xmm2
    movdqa  xmm2, xmmword ptr [rip + .LCPI0_1]
    aesenc  xmm2, xmm3
    movdqa  xmm3, xmmword ptr [rip + .LCPI0_2]
    jmp     .LBB0_10
.LBB0_5:
    shr     rax
    shl     rdx, 4
    mov     rcx, rsi
    sub     rcx, rdx
    movdqa  xmm1, xmmword ptr [rip + .LCPI0_0]
    movdqa  xmm2, xmmword ptr [rip + .LCPI0_1]
    movdqa  xmm3, xmmword ptr [rip + .LCPI0_2]
    movdqa  xmm6, xmmword ptr [rip + .LCPI0_3]
    movdqa  xmm5, xmmword ptr [rip + .LCPI0_4]
    movdqa  xmm4, xmmword ptr [rip + .LCPI0_5]
.LBB0_6:
    movdqu  xmm7, xmmword ptr [rdi]
    movdqu  xmm8, xmmword ptr [rdi + 16]
    movdqu  xmm9, xmmword ptr [rdi + rcx - 16]
    movdqu  xmm10, xmmword ptr [rdi + rcx]
    add     rdi, 32
    aesenc  xmm1, xmm7
    aesenc  xmm6, xmm8
    aesenc  xmm2, xmm9
    aesenc  xmm5, xmm10
    aesenc  xmm3, xmm7
    aesenc  xmm4, xmm8
    aesenc  xmm3, xmm9
    aesenc  xmm4, xmm10
    dec     rax
    jne     .LBB0_6
    aesenc  xmm0, xmm6
    aesenc  xmm0, xmm5
    aesenc  xmm0, xmm4
.LBB0_10:
    aesenc  xmm0, xmm1
    aesenc  xmm0, xmm2
    aesenc  xmm0, xmm3
    movq    rcx, xmm0
    pshufd  xmm0, xmm0, 238
    movq    rax, xmm0
    mul     rcx
    xor     esi, edx
    xor     esi, eax
    mov     dword ptr [r8], esi
    ret
*/

```

// FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla: the 100% FREE lookuper, last update: 2026-Aug-09, Kaze (sanmayce@sanmayce.com).]

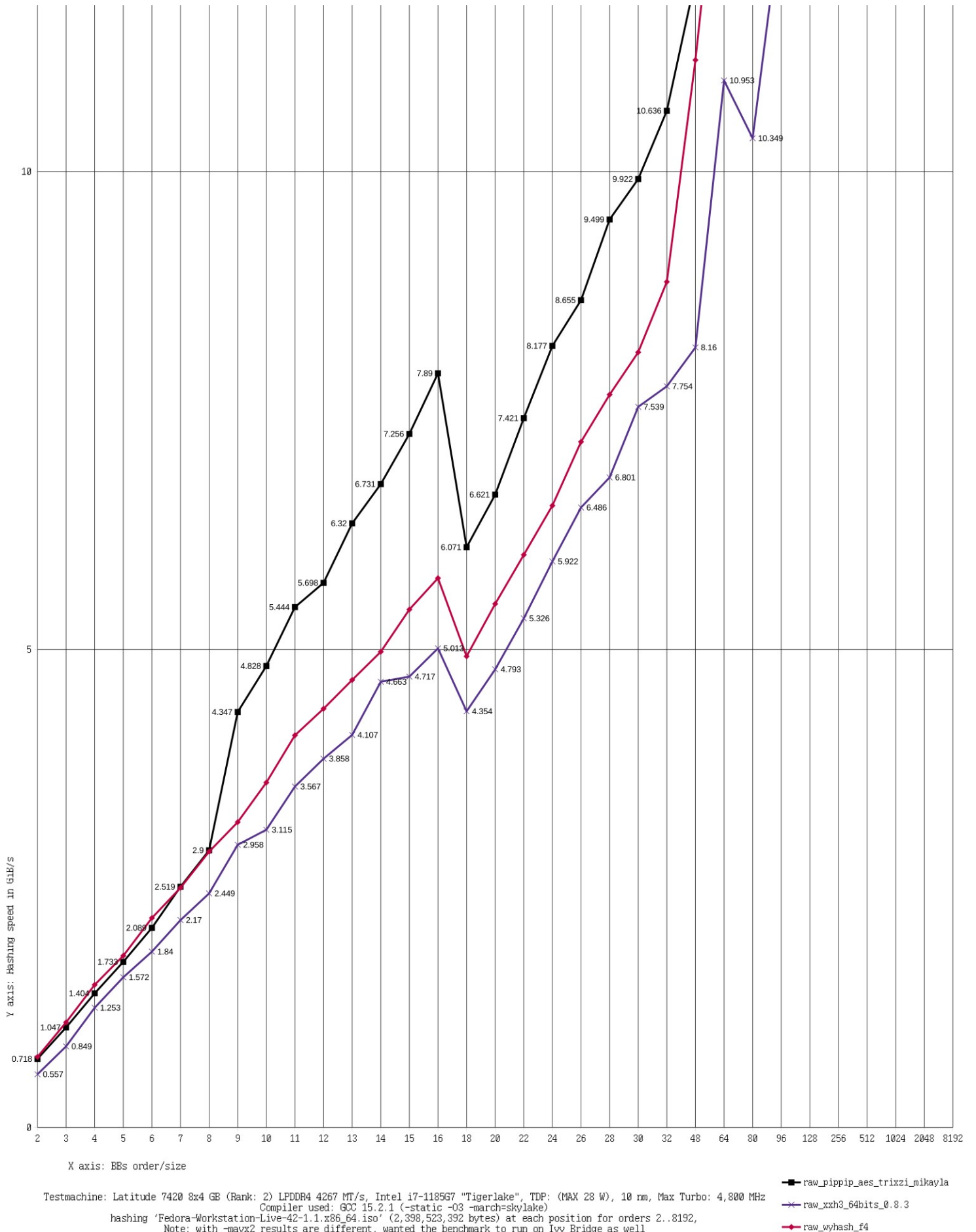

```
#define _PADr_KAZE(x, n) ( ((x) << (n)) >> (n) )
#define _PAD_KAZE(x, n) ( ((x) << (n)) )
static inline uint64_t fold64(uint64_t A, uint64_t B) {
    #if defined(__GNUC__) || defined(__clang__)
        uint128_t r = (__uint128_t)A * B;
        return (uint64_t)r ^ (uint64_t)(r >> 64);
    #else
        uint64_t hash64 = A ^ B;
        hash64 *= 1099511628211; //591798841;
        return hash64;
    #endif
}
// Actually 'FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla_FOX'
void FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla(const char *str, size_t wrdlen, uint32_t seed, void *output) {
    __m128i chunkA; __m128i chunkB; __m128i stateMIX;
    uint64_t hashLH; uint64_t hashRH;
    stateMIX = __mm_set_epi32( (uint32_t)wrdlen ^ seed );
    __m128i stateA = __mm_set_epi64x(0x6c62272e07bb0142, 0x9e3779b97f4a7c15);
    __m128i stateB = __mm_set_epi64x(0x6A09E667F3BCC908, 0xBB67AE8584CAA73B);
    __m128i stateC = __mm_set_epi64x(0x3C6EF372FE94F82B, 0x428A2F98D728AE22);
    if (wrdlen > 8) {
        size_t Cycles, NDhead;
        if (wrdlen > 16) {
            Cycles = ((wrdlen - 1) >> 5) + 1;
            NDhead = wrdlen - (Cycles << 4);
            if (Cycles & 1) {
                #pragma nounroll
                for(; Cycles--; str += 16) {
                    chunkA = __mm_loadu_si128((__m128i *) (str));
                    chunkB = __mm_loadu_si128((__m128i *) (str+NDhead));
                    stateA = __mm_aesenc_si128(stateA, chunkA);
                    stateB = __mm_aesenc_si128(stateB, chunkB);
                    stateC = __mm_aesenc_si128(stateC, chunkA);
                    stateC = __mm_aesenc_si128(stateC, chunkB);
                }
            } else {
                Cycles = Cycles >> 1;
                __m128i stateA2 = __mm_set_epi64x(0x9e3779b97f4a7c15, 0x6c62272e07bb0142);
                __m128i stateB2 = __mm_set_epi64x(0xBB67AE8584CAA73B, 0x6A09E667F3BCC908);
                __m128i stateC2 = __mm_set_epi64x(0x428A2F98D728AE22, 0x3C6EF372FE94F82B);
                #pragma nounroll
                for(; Cycles--; str += 32) {
                    __mm_prefetch(str, __MM_HINT_T0);
                    __mm_prefetch(str+NDhead, __MM_HINT_T0);
                    chunkA = __mm_loadu_si128((__m128i *) (str));
                    __m128i chunkA2 = __mm_loadu_si128((__m128i *) (str+16));
                    stateA = __mm_aesenc_si128(stateA, chunkA);
                    stateA2 = __mm_aesenc_si128(stateA2, chunkA2);
                    chunkB = __mm_loadu_si128((__m128i *) (str+NDhead));
                    __m128i chunkB2 = __mm_loadu_si128((__m128i *) (str+NDhead+16));
                    stateB = __mm_aesenc_si128(stateB, chunkB);
                    stateB2 = __mm_aesenc_si128(stateB2, chunkB2);
                    stateC = __mm_aesenc_si128(stateC, chunkA);
                    stateC2 = __mm_aesenc_si128(stateC2, chunkA2);
                    stateC = __mm_aesenc_si128(stateC, chunkB);
                    stateC2 = __mm_aesenc_si128(stateC2, chunkB2);
                }
                stateMIX = __mm_aesenc_si128(stateMIX, stateA2);
                stateMIX = __mm_aesenc_si128(stateMIX, stateB2);
                stateMIX = __mm_aesenc_si128(stateMIX, stateC2);
            }
        } else { // 9..16
            NDhead = wrdlen - (1 << 3);
            hashLH = (*(uint64_t *) (str));
            hashRH = (*(uint64_t *) (str+NDhead));
            chunkA = __mm_set_epi64x(hashLH, hashLH);
            chunkB = __mm_set_epi64x(hashRH, hashRH);
            chunkA = __mm_xor_si128(chunkA, stateMIX);
            chunkB = __mm_xor_si128(chunkB, stateMIX);
            stateA = __mm_aesenc_si128(stateA, chunkA);
            stateB = __mm_aesenc_si128(stateB, chunkB);
            stateC = __mm_aesenc_si128(stateC, chunkA);
            stateC = __mm_aesenc_si128(stateC, chunkB);
        }
    } else {
        hashLH = _PADr_KAZE(*(uint64_t *) (str+0), (8-wrdlen) << 3); // wrdlen == 0 is not handled since <<64 is undefined
        hashRH = _PAD_KAZE(*(uint64_t *) (str+0), (8-wrdlen) << 3);
        chunkA = __mm_set_epi64x(hashLH, hashRH);
        chunkB = __mm_set_epi64x(hashRH, hashLH);
        stateA = __mm_aesenc_si128(stateA, chunkA);
        stateB = __mm_aesenc_si128(stateB, chunkB);
        stateC = __mm_aesenc_si128(stateC, chunkA);
        stateC = __mm_aesenc_si128(stateC, chunkB);
    }
    stateMIX = __mm_aesenc_si128(stateMIX, stateA);
    stateMIX = __mm_aesenc_si128(stateMIX, stateB);
    stateMIX = __mm_aesenc_si128(stateMIX, stateC);
    #ifdef eXdupe
        __mm_storeu_si128((__m128i *) output, stateMIX); // For eXdupe
    #else
        uint64_t result[2];
        __mm_storeu_si128((__m128i *) result, stateMIX);
        uint64_t hash64 = fold64(result[0], result[1]);
        *(uint32_t *) output = (uint32_t) hash64 ^ wrdlen;
    #endif
}

```

```
#define_PADr_KAZE(x, n) ( ((x) << (n)) >> (n) )
#define_PAD_KAZE(x, n) ( ((x) << (n)) )
static inline uint64_t fold64(uint64_t A, uint64_t B) {
    #if defined(__GNUC__) || defined(__clang__)
        uint128_t r = (__uint128_t)A * B;
        return (uint64_t)r ^ (uint64_t)(r >> 64);
    #else
        uint64_t hash64 = A ^ B;
        hash64 *= 1099511628211; //591798841;
        return hash64;
    #endif
}

// Actually 'FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla_FOX'
void FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla (const char *str, size_t wrdlen, uint32_t seed, void *output) {
    _m128i chunkA; _m128i chunkB; _m128i stateMIX;
    uint64_t hashLH; uint64_t hashRH;
    stateMIX = _mm_set1_epi32( (uint32_t)wrdlen ^ seed );
    _m128i stateA = _mm_set_epi64x(0x6c62272e07bb0142, 0x9e3779b97f4a7c15);
    _m128i stateB = _mm_set_epi64x(0x6A09E667F3BCC908, 0xBB67AE8584CAA73B);
    _m128i stateC = _mm_set_epi64x(0x3C6EF372FE94F82B, 0x428A2F98D728AE22);
    if (wrdlen > 8) {
        size_t Cycles, NDhead;
        if (wrdlen > 16) {
            Cycles = ((wrdlen - 1) >> 5) + 1;
            NDhead = wrdlen - (Cycles << 4);
            if (Cycles & 1) {
                #pragma nounroll
                for (; Cycles--; str += 16) {
                    chunkA = _mm_loadu_si128((__m128i *) (str));
                    chunkB = _mm_loadu_si128((__m128i *) (str+NDhead));
                    stateA = _mm_aesenc_si128(stateA, chunkA);
                    stateB = _mm_aesenc_si128(stateB, chunkB);
                    stateC = _mm_aesenc_si128(stateC, chunkA);
                    stateC = _mm_aesenc_si128(stateC, chunkB);
                }
            } else {
                Cycles = Cycles >> 1;
                _m128i stateA2 = _mm_set_epi64x(0x9e3779b97f4a7c15, 0x6c62272e07bb0142);
                _m128i stateB2 = _mm_set_epi64x(0xBB67AE8584CAA73B, 0x6A09E667F3BCC908);
                _m128i stateC2 = _mm_set_epi64x(0x428A2F98D728AE22, 0x3C6EF372FE94F82B);
                #pragma nounroll
                for (; Cycles--; str += 32) {
                    _mm_prefetch(str, _MM_HINT_T0);
                    _mm_prefetch(str+NDhead, _MM_HINT_T0);
                    chunkA = _mm_loadu_si128((__m128i *) (str));
                    _m128i chunkA2 = _mm_loadu_si128((__m128i *) (str+16));
                    stateA = _mm_aesenc_si128(stateA, chunkA);
                    stateA2 = _mm_aesenc_si128(stateA2, chunkA2);
                    chunkB = _mm_loadu_si128((__m128i *) (str+NDhead));
                    _m128i chunkB2 = _mm_loadu_si128((__m128i *) (str+NDhead+16));
                    stateB = _mm_aesenc_si128(stateB, chunkB);
                    stateB2 = _mm_aesenc_si128(stateB2, chunkB2);
                    stateC = _mm_aesenc_si128(stateC, chunkA);
                    stateC2 = _mm_aesenc_si128(stateC2, chunkA2);
                    stateC = _mm_aesenc_si128(stateC, chunkB);
                    stateC2 = _mm_aesenc_si128(stateC2, chunkB2);
                }
                stateMIX = _mm_aesenc_si128(stateMIX, stateA2);
                stateMIX = _mm_aesenc_si128(stateMIX, stateB2);
                stateMIX = _mm_aesenc_si128(stateMIX, stateC2);
            }
        } else { // 9..16
            NDhead = wrdlen - (1 << 3);
            hashLH = (*(uint64_t *) (str));
            hashRH = (*(uint64_t *) (str+NDhead));
            chunkA = _mm_set_epi64x(hashLH, hashLH);
            chunkB = _mm_set_epi64x(hashRH, hashRH);
            chunkA = _mm_xor_si128(chunkA, stateMIX);
            chunkB = _mm_xor_si128(chunkB, stateMIX);
            stateA = _mm_aesenc_si128(stateA, chunkA);
            stateB = _mm_aesenc_si128(stateB, chunkB);
            stateC = _mm_aesenc_si128(stateC, chunkA);
            stateC = _mm_aesenc_si128(stateC, chunkB);
        }
        stateMIX = _mm_aesenc_si128(stateMIX, stateA);
        stateMIX = _mm_aesenc_si128(stateMIX, stateB);
        stateMIX = _mm_aesenc_si128(stateMIX, stateC);
        #ifdef EXDUPE
            _mm_storeu_si128((__m128i *) output, stateMIX);
        #else
            uint64_t result[2];
            _mm_storeu_si128((__m128i *) result, stateMIX);
            uint64_t hash64 = fold64(result[0], result[1]);
            *(uint32_t *) output = (uint32_t) hash64 ^ wrdlen;
        #endif
    }
}
```





Testfile: Fedora-Workstation-Live-42-1.1.x86_64.iso (2,398,523,392 bytes)

Since the output is both CSV (COMMA-separated) and TSV (TAB-separated) I converted TAB to SPC in order to ease the grep search:

...

[sanmayce@judjeto hashBBs_ver10_Fedora]\$ grep " 4," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt

Listing: **FNW1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla_FOX**; Update: 2026-Aug-10, Kaze(sanmayce@sanmayce.com);

```
Pippip_AES_TriXZi_Mikayla, 4, 2398523389, 1352622127, 1,045,901,262, 45.176, 0.198, 1.404
wyhash f4, 4, 2398523389, 1352625729, 1,045,897,660, 52.491, 0.170, 1.493
XXH3_64bits 0.8.3, 4, 2398523389, 1352586881, 1,045,936,508, 54.860, 0.163, 1.253
```

```
[sanmayce@djudjeto hashBBs_ver10_Fedora]$ grep " 8," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt
Pippip_AES_TriXZi_Mikayla, 8, 2398523385, 1630426662, 768,096,723, 45.409, 0.394, 2.900
wyhash f4, 8, 2398523385, 1630416504, 768,106,881, 52.703, 0.339, 2.886
XXH3_64bits 0.8.3, 8, 2398523385, 1630426332, 768,097,053, 54.238, 0.329, 2.449
```

```
[sanmayce@djudjeto hashBBs_ver10_Fedora]$ grep " 16," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt
Pippip_AES_TriXZi_Mikayla, 16, 2398523377, 1636023415, 762,499,962, 38.993, 0.917, 7.890
wyhash f4, 16, 2398523377, 1636017018, 762,506,359, 52.753, 0.678, 5.747
XXH3_64bits 0.8.3, 16, 2398523377, 1636042386, 762,480,991, 53.250, 0.671, 5.013
```

```
[sanmayce@djudjeto hashBBs_ver10_Fedora]$ grep " 32," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt
Pippip_AES_TriXZi_Mikayla, 32, 2398523361, 1641852738, 756,670,623, 49.017, 1.458, 10.636
wyhash f4, 32, 2398523361, 1641842027, 756,681,334, 61.103, 1.170, 8.847
XXH3_64bits 0.8.3, 32, 2398523361, 1641854953, 756,668,408, 63.389, 1.128, 7.754
```

```
[sanmayce@djudjeto hashBBs_ver10_Fedora]$ grep " 64," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt
Pippip_AES_TriXZi_Mikayla, 64, 2398523329, 1646164301, 752,359,028, 59.174, 2.416, 16.261
wyhash f4, 64, 2398523329, 1646117589, 752,405,740, 75.084, 1.904, 13.974
XXH3_64bits 0.8.3, 64, 2398523329, 1646133185, 752,390,144, 85.474, 1.673, 10.953
```

```
[sanmayce@djudjeto hashBBs_ver10_Fedora]$ grep " 512," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt
Pippip_AES_TriXZi_Mikayla, 512, 2398522881, 1655001920, 743,503,061, 160.196, 7.139, 45.477
wyhash f4, 512, 2398522881, 1655000161, 743,522,720, 312.229, 3.663, 21.700
XXH3_64bits 0.8.3, 512, 2398522881, 1655001212, 743,521,669, 175.538, 6.515, 29.128
```

```
[sanmayce@djudjeto hashBBs_ver10_Fedora]$ grep " 2048," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt
Pippip_AES_TriXZi_Mikayla, 2048, 2398521345, 1673907122, 724,614,223, 374.689, 12.210, 45.485
wyhash f4, 2048, 2398521345, 1673876063, 724,645,282, 478.317, 9.564, 22.801
XXH3_64bits 0.8.3, 2048, 2398521345, 1673886241, 724,635,104, 404.159, 11.319, 39.968
```

```
[sanmayce@djudjeto hashBBs_ver10_Fedora]$ grep " 8192," /home/sanmayce/Downloads/aug-08/hashBBs_ver10_Fedora/results_7420_noTAB.txt
Pippip_AES_TriXZi_Mikayla, 8192, 2398515201, 1694715910, 703,799,291, 748.655, 24.443, 42.894
wyhash f4, 8192, 2398515201, 1694711049, 703,804,152, 1092.971, 16.743, 24.122
XXH3_64bits 0.8.3, 8192, 2398515201, 1694695692, 703,819,509, 725.755, 25.214, 45.960
'''
```

Let us see which hasher gives minimum collisions for granularities (4,8,16,32,64,512,2048,8192), after all we use 32bit hash table (keys:slots = 2,398,523,392:2^32 = 0.55 = ~1:2):

'''

```
Pippip_AES_TriXZi_Mikayla, 4, 2398523389, 1352622127, 1,045,901,262, 45.176, 0.198, 1.404
Pippip_AES_TriXZi_Mikayla, 8, 2398523385, 1630426662, 768,096,723, 45.409, 0.394, 2.900
Pippip_AES_TriXZi_Mikayla, 16, 2398523377, 1636023415, 762,499,962, 38.993, 0.917, 7.890
Pippip_AES_TriXZi_Mikayla, 32, 2398523361, 1641852738, 756,670,623, 49.017, 1.458, 10.636
Pippip_AES_TriXZi_Mikayla, 64, 2398523329, 1646164301, 752,359,028, 59.174, 2.416, 16.261
Pippip_AES_TriXZi_Mikayla, 512, 2398522881, 1655019820, 743,503,061, 160.196, 7.139, 45.477
Pippip_AES_TriXZi_Mikayla, 2048, 2398521345, 1673907122, 724,614,223, 374.689, 12.210, 45.485
Pippip_AES_TriXZi_Mikayla, 8192, 2398515201, 1694715910, 703,799,291, 748.655, 24.443, 42.894 ! prefetching was commented out !
```

1,045,901,262+768,096,723+762,499,962+756,670,623+752,359,028+743,503,061+724,614,223+703,799,291= 6,257,444,173 collisions

```
wyhash f4, 4, 2398523389, 1352625729, 1,045,897,660, 52.491, 0.170, 1.493
wyhash f4, 8, 2398523385, 1630416504, 768,106,881, 52.703, 0.339, 2.886
wyhash f4, 16, 2398523377, 1636017018, 762,506,359, 52.753, 0.678, 5.747
wyhash f4, 32, 2398523361, 1641842027, 756,681,334, 61.103, 1.170, 8.847
wyhash f4, 64, 2398523329, 1646117589, 752,405,740, 75.084, 1.904, 13.974
wyhash f4, 512, 2398522881, 1655000161, 743,522,720, 312.229, 3.663, 21.700
wyhash f4, 2048, 2398521345, 1673876063, 724,645,282, 478.317, 9.564, 22.801
wyhash f4, 8192, 2398515201, 1694711049, 703,804,152, 1092.971, 16.743, 24.122
```

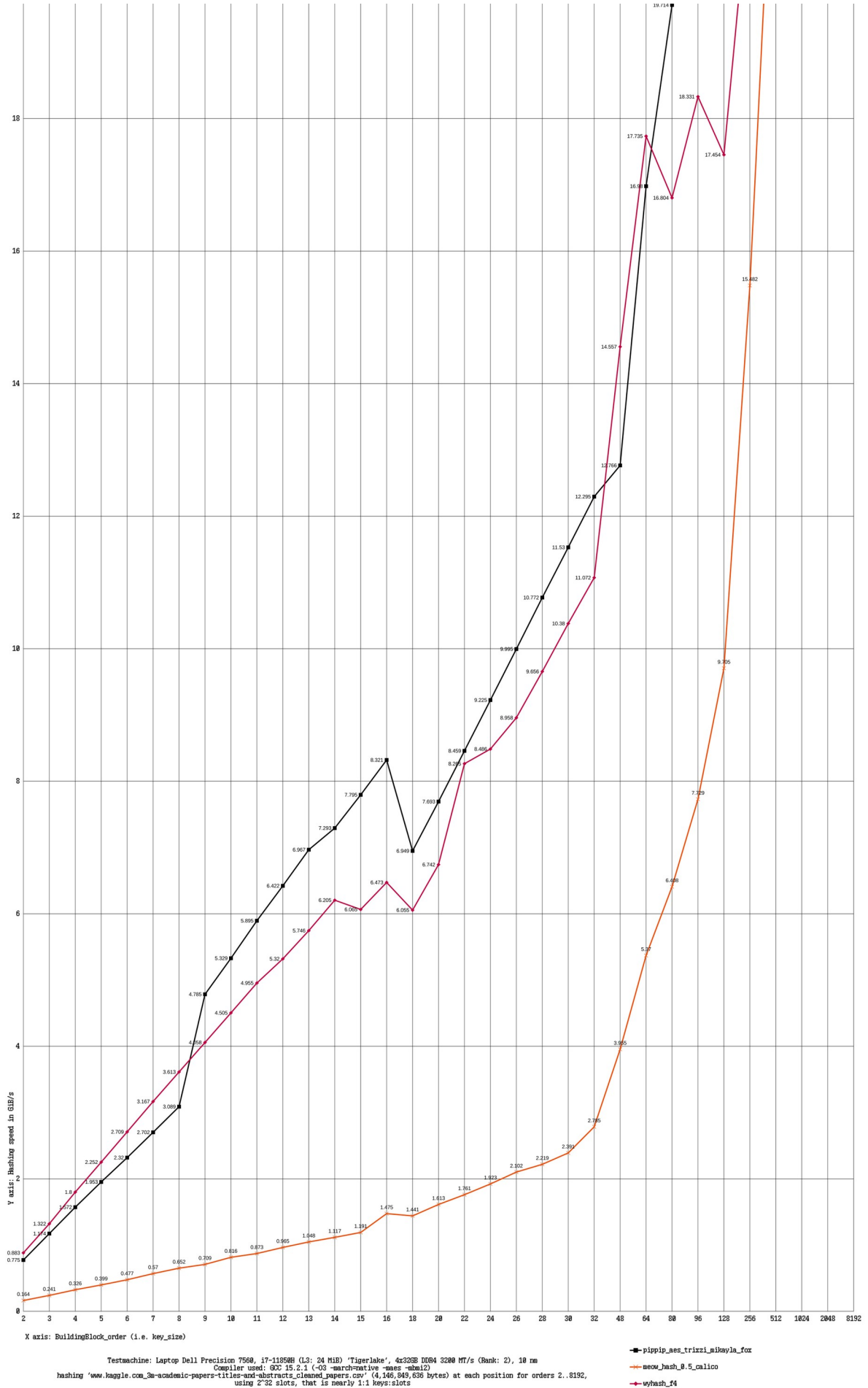
1,045,897,660+768,106,881+762,506,359+756,681,334+752,405,740+743,522,720+724,645,282+703,804,152= 6,257,570,128 collisions, (or 6,257,570,128-6,257,444,173= 125,955 more)

```
XXH3_64bits 0.8.3, 4, 2398523389, 1352586881, 1,045,936,508, 54.860, 0.163, 1.253
XXH3_64bits 0.8.3, 8, 2398523385, 1630426332, 768,097,053, 54.238, 0.329, 2.449
XXH3_64bits 0.8.3, 16, 2398523377, 1636042386, 762,480,991, 53.250, 0.671, 5.013
XXH3_64bits 0.8.3, 32, 2398523361, 1641854953, 756,668,408, 63.389, 1.128, 7.754
XXH3_64bits 0.8.3, 64, 2398523329, 1646133185, 752,390,144, 85.474, 1.673, 10.953
XXH3_64bits 0.8.3, 512, 2398522881, 1655001212, 743,521,669, 175.538, 6.515, 29.128
XXH3_64bits 0.8.3, 2048, 2398521345, 1673886241, 724,635,104, 404.159, 11.319, 39.968
XXH3_64bits 0.8.3, 8192, 2398515201, 1694695692, 703,819,509, 725.755, 25.214, 45.960
```

1,045,936,508+768,097,053+762,480,991+756,668,408+752,390,144+743,521,669+724,635,104+703,819,509= 6,257,549,386 collisions, (or 6,257,549,386-6,257,444,173= 105,213 more)

'''

So, on my daily working donkey (Dell 7420), Pippip outperforms the two superfast hashers in speed and collision departments.



Testfile: www.kaggle.com/3m-academic-papers-titles-and-abstracts_cleaned_papers.csv (4,146,849,636 bytes)
 Since the output is both CSV (COMMA-separated) and TSV (TAB-separated) I converted TAB to SPC in order to ease the grep search:

```
[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 2," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
```

Listing: **FNV1A_Pippip_Yurii_000_128bit_AES_TriXZi_Mikayla_FOX**; Update: 2026-Aug-10, Kaze(sanmayce@sanmayce.com);

```

MeowHash, 2, 4146849635, 16399, 4146833236, 41.216, 0.187, 0.193
Pippip_AES_TriXZi_Mikayla, 2, 4146849635, 16399, 4146833236, 11.856, 0.651, 0.775
wyhash f4, 2, 4146849635, 16399, 4146833236, 9.814, 0.787, 0.883

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 3," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 3, 4146849634, 507862, 4146341772, 46.137, 0.251, 0.289
Pippip_AES_TriXZi_Mikayla, 3, 4146849634, 507868, 4146341766, 14.035, 0.826, 1.174
wyhash f4, 3, 4146849634, 507862, 4146341772, 11.905, 0.973, 1.322

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 4," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 4, 4146849633, 4661520, 4142188113, 55.994, 0.276, 0.369
Pippip_AES_TriXZi_Mikayla, 4, 4146849633, 4661388, 4142188245, 19.195, 0.805, 1.572
wyhash f4, 4, 4146849633, 4661498, 4142188135, 18.363, 0.841, 1.800

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 5," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 5, 4146849632, 18384187, 4128465445, 72.051, 0.268, 0.482
Pippip_AES_TriXZi_Mikayla, 5, 4146849632, 18384209, 4128465423, 25.565, 0.755, 1.953
wyhash f4, 5, 4146849632, 18384390, 4128465242, 25.657, 0.753, 2.252

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 6," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 6, 4146849631, 46720333, 4100129298, 89.521, 0.259, 0.578
Pippip_AES_TriXZi_Mikayla, 6, 4146849631, 46720298, 4100129333, 34.113, 0.679, 2.320
wyhash f4, 6, 4146849631, 46720973, 4100128658, 34.589, 0.670, 2.709

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 7," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 7, 4146849630, 90355219, 4056494411, 113.680, 0.238, 0.675
Pippip_AES_TriXZi_Mikayla, 7, 4146849630, 90355514, 4056494116, 42.543, 0.635, 2.702
wyhash f4, 7, 4146849630, 90353268, 4056496362, 43.835, 0.617, 3.167

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 8," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 8, 4146849629, 149662516, 3997187113, 138.538, 0.223, 0.771
Pippip_AES_TriXZi_Mikayla, 8, 4146849629, 149660856, 3997188773, 49.229, 0.628, 3.089
wyhash f4, 8, 4146849629, 149664375, 3997185254, 51.226, 0.603, 3.613

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 9," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 9, 4146849628, 227582427, 3919267201, 161.745, 0.215, 0.868
Pippip_AES_TriXZi_Mikayla, 9, 4146849628, 227586153, 3919263475, 48.198, 0.721, 4.785
wyhash f4, 9, 4146849628, 227586151, 3919263477, 56.626, 0.614, 4.058

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 10," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 10, 4146849627, 325944646, 3820904981, 183.351, 0.211, 0.964
Pippip_AES_TriXZi_Mikayla, 10, 4146849627, 325946837, 3820902790, 50.963, 0.758, 5.329
wyhash f4, 10, 4146849627, 325945511, 3820904116, 60.841, 0.635, 4.505

[sanmayce@djudgeto Golden_2026-Aug-09]$ grep " 11," /home/sanmayce/Downloads/Golden_2026-Aug-09/7560_3_tigerlake_4GB.csv
MeowHash, 11, 4146849626, 444142847, 3702706779, 199.146, 0.213, 1.047
Pippip_AES_TriXZi_Mikayla, 11, 4146849626, 444151929, 3702697697, 52.966, 0.802, 5.895
wyhash f4, 11, 4146849626, 444145232, 3702704394, 64.157, 0.662, 4.955
'''

We get the total of 4th column (the unique hashes):
'''
MeowHash, 2, 4146849635, 16399, 4146833236, 41.216, 0.187, 0.193
MeowHash, 3, 4146849634, 507862, 4146341772, 46.137, 0.251, 0.289
MeowHash, 4, 4146849633, 4661520, 4142188113, 55.994, 0.276, 0.369
MeowHash, 5, 4146849632, 18384187, 4128465445, 72.051, 0.268, 0.482
MeowHash, 6, 4146849631, 46720333, 4100129298, 89.521, 0.259, 0.578
MeowHash, 7, 4146849630, 90355219, 4056494411, 113.680, 0.238, 0.675
MeowHash, 8, 4146849629, 149662516, 3997187113, 138.538, 0.223, 0.771
MeowHash, 9, 4146849628, 227582427, 3919267201, 161.745, 0.215, 0.868
MeowHash, 10, 4146849627, 325944646, 3820904981, 183.351, 0.211, 0.964
MeowHash, 11, 4146849626, 444142847, 3702706779, 199.146, 0.213, 1.047

16399+507862+4661520+18384187+46720333+90355219+149662516+227582427+325944646+444142847= 1,307,977,956 dispersion (the-bigger-the-better),
1,307,991,451-1,307,977,956= 13,495 worse than Pippip

Pippip_AES_TriXZi_Mikayla, 2, 4146849635, 16399, 4146833236, 11.856, 0.651, 0.775
Pippip_AES_TriXZi_Mikayla, 3, 4146849634, 507868, 4146341766, 14.035, 0.826, 1.174
Pippip_AES_TriXZi_Mikayla, 4, 4146849633, 4661388, 4142188245, 19.195, 0.805, 1.572
Pippip_AES_TriXZi_Mikayla, 5, 4146849632, 18384209, 4128465423, 25.565, 0.755, 1.953
Pippip_AES_TriXZi_Mikayla, 6, 4146849631, 46720298, 4100129333, 34.113, 0.679, 2.320
Pippip_AES_TriXZi_Mikayla, 7, 4146849630, 90355514, 4056494116, 42.543, 0.635, 2.702
Pippip_AES_TriXZi_Mikayla, 8, 4146849629, 149660856, 3997188773, 49.229, 0.628, 3.089
Pippip_AES_TriXZi_Mikayla, 9, 4146849628, 227586153, 3919263475, 48.198, 0.721, 4.785
Pippip_AES_TriXZi_Mikayla, 10, 4146849627, 325946837, 3820902790, 50.963, 0.758, 5.329
Pippip_AES_TriXZi_Mikayla, 11, 4146849626, 444151929, 3702697697, 52.966, 0.802, 5.895

16399+507868+4661388+18384209+46720298+90355514+149660856+227586153+325946837+444151929= 1,307,991,451 dispersion (the-bigger-the-better)

wyhash f4, 2, 4146849635, 16399, 4146833236, 9.814, 0.787, 0.883
wyhash f4, 3, 4146849634, 507862, 4146341772, 11.905, 0.973, 1.322
wyhash f4, 4, 4146849633, 4661498, 4142188135, 18.363, 0.841, 1.800
wyhash f4, 5, 4146849632, 18384390, 4128465242, 25.657, 0.753, 2.252
wyhash f4, 6, 4146849631, 46720973, 4100128658, 34.589, 0.670, 2.709
wyhash f4, 7, 4146849630, 90353268, 4056496362, 43.835, 0.617, 3.167
wyhash f4, 8, 4146849629, 149664375, 3997185254, 51.226, 0.603, 3.613
wyhash f4, 9, 4146849628, 227586151, 3919263477, 56.626, 0.614, 4.058
wyhash f4, 10, 4146849627, 325945511, 3820904116, 60.841, 0.635, 4.505
wyhash f4, 11, 4146849626, 444145232, 3702704394, 64.157, 0.662, 4.955

16399+507862+4661498+18384390+46720973+90353268+149664375+227586151+325945511+444145232= 1,307,985,659 dispersion (the-bigger-the-better),
1,307,991,451-1,307,985,659= 5,792 worse than Pippip
'''

```


Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping



Fastest Hash for lookuping

